université
PARIS-SACLAY

RAPPORT DE TP

VISION ROBOTIQUE

Auteur :

Gaëtan BOUCHY
Thomas ABOU JAOUDE

Enseignant :

M. LERME

À Orsay, le 05/02/2022

université
PARIS-SACLAY

Sommaire

Sommaire 2
1. Seam Carving Basique 3
Introduction 3
1.1 Charger l'image I en mémoire et calculer l'image E_1 qui est la norme L_1 du gradient de I 4
1.2 Construire le graphe G à l'aide de l'image E_1 5
1.3 Construire l'image binaire qui est une partition de G 9
1.4 Chercher la frontière entre S et T, la supprimer et recaller les deux parties 10
1.5 Recommencer à l'étape 3 (« Construire le graphe G à l'aide de l'image E_1 ») jusqu'à atteindre la longueur d'image voulue 12
1.6 Analyse de la complexité et du temps d'exécution de l'algorithme 13
1.7 Analyse des résultats obtenus 14
1.7.1 Le banc 14
1.7.2 La plage 19
1.7.3 L'oiseau 22
2. Seam carving amélioré 25
Introduction 25
2.1 Choix des paramètres $\boldsymbol{\lambda 1}$ et $\boldsymbol{\lambda 2}$ 25
3 Annexe (code Matlab) 26

université
PARIS-SACLAY

1. Seam Carving Basique

Introduction

L'algorithme de Seam Carving est un algorithme innovant pour le redimensionnement d'images, qui permet de réduire ou d'agrandir l'image sans altérer les proportions des objets importants présents dans l'image. Ce processus est réalisé en identifiant les « coutures » les plus minces dans l'image et en les enlevant ou en les ajoutant pour atteindre la taille souhaitée. Contrairement aux méthodes traditionnelles de redimensionnement d'images qui peuvent affecter la qualité de l'image en la distordant, Seam Carving préserve la qualité de l'image tout en modifiant sa taille.

Afin de réaliser cet algorithme sur Matlab, nous avons dû suivre un certain nombre d'étapes que nous allons détailler :

- Charger l'image I en mémoire
- Calculer l'image E_1 qui est la norme L_1 du gradient de I
- Construire le graphe G à l'aide de l'image E_1
- Calculer |f*| = |C*| ainsi que C = (S,T)
- Construire l'image binaire qui est une partition de G
- Chercher la frontière entre S et T, la supprimer et recaller les deux parties
- Recommencer à l'étape 3 (« Construire le graphe G à l'aide de l'image E_1 ») jusqu'à atteindre la longueur d'image voulue

université
PARIS-SACLAY

1.1 Charger l'image I en mémoire et calculer l'image E_1 qui est la norme L_1 du gradient de I

Ce code MATLAB charge une image appelée « **banc.jpg** » et la stocke dans la variable « **I** ». Si l'image est en couleurs (3 dimensions), il la convertit en une image en niveaux de gris en utilisant la fonction « **rgb2gray** ».

Ensuite, la fonction « **size** » est utilisée pour obtenir la hauteur et la largeur de l'image et les stocker dans les variables « **hauteur** » et « **largeur** ».

La fonction « **imgradientxy** » est utilisée pour calculer les gradients selon l'axe X et l'axe Y de l'image en niveaux de gris et les stocker dans les variables « **gradientX** » et « **gradientY** ».

Enfin, la variable « **E1** » est définie comme la somme absolue des gradientX et gradientY, ce qui représente la norme L1 du gradient de l'image. Cela sera utilisé plus tard pour déterminer les coutures dans l'algorithme de Seam Carving.

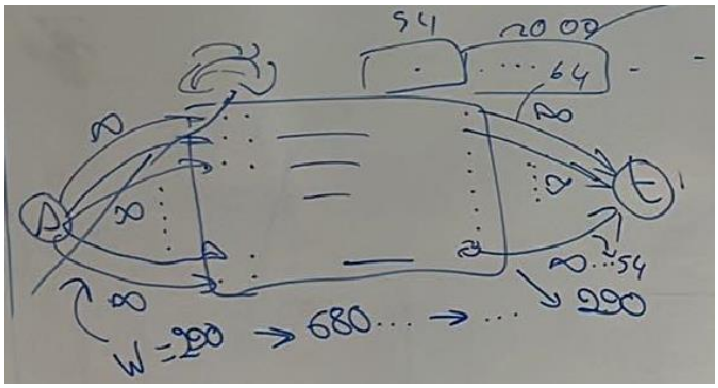
```
I = imread('banc.jpg'); % Charger l'image
if ndims(I) == 3
    I = rgb2gray(I); % Convertir en niveaux de gris
end

[hauteur, largeur] = size(I);
% Calcul de la norme L1 du gradient
[gradientX,gradientY] = imgradientxy(I);
E1 = abs(gradientX) + abs(gradientY);
```

Figure 1 : Code pour charger l'image I en mémoire et calculer l'image E_1 qui est la norme L_1 du gradient de I

1.2 Construire le graphe G à l'aide de l'image E_1

Afin de construire le graphe G à l'aide de l'image E_1, nous souhaitons réaliser ces deux schémas :



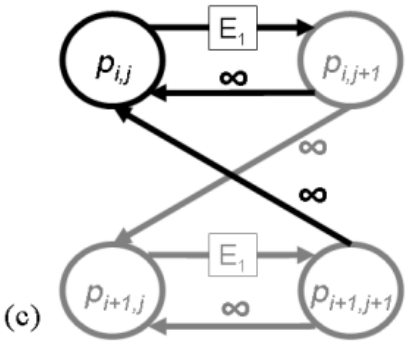


Figure 2 : Graphiques permettant la construction du graphe G

La figure à gauche montre la source S reliée à tous les pixels du bord gauche de l'image. Le poids allant de la source S aux pixels du bord gauche vaut l'infini c'est-à-dire une très grande valeur :

```
boucle = 1;

compteurBoucle = 0;

for colonne = 1:hauteur

    compteurBoucle = compteurBoucle + 1;

    source_nodes_idx(compteurBoucle) = boucle;

    destination_nodes_idx(compteurBoucle) = (boucle+1)+((colonne-1)*largeur);

    weights(compteurBoucle) = 2000000;

end
```

Figure 3 : Code permettant de lier la source S à tous les pixels du bord gauche en leur assignant un poids de 20000000 représentant l'infini

Après cela, nous avons traité les différentes liaisons qu'il y a entre chaque pixel comme le montre l'image de droite :
- Le pixel $p_{i,j}$ est lié au pixel $p_{i,j+1}$ par un poids E_1 calculé précédemment
- Le pixel $p_{i,j+1}$ est lié au pixel $p_{i,j}$ par un poids ∞ soit 2 000 000 dans notre code
- Le pixel $p_{i,j+1}$ est lié au pixel $p_{i+1,j}$ par un poids ∞ soit 2 000 000 dans notre code
- Le pixel $p_{i+1,j+1}$ est lié au pixel $p_{i,j}$ par un poids ∞ soit 2 000 000 dans notre code

```
for colonne = 1:hauteur-1

        if colonne > 1
           boucle = boucle + 1;
        end

        for ligne = 1:largeur-1

            boucle = boucle + 1;

            compteurBoucle = compteurBoucle + 1;
            source_nodes_idx(compteurBoucle) = boucle;
            destination_nodes_idx(compteurBoucle) = boucle+1;
            weights(compteurBoucle) = E1(colonne,ligne);

            compteurBoucle = compteurBoucle + 1;
            source_nodes_idx(compteurBoucle) = boucle+1;
            destination_nodes_idx(compteurBoucle) = boucle;
            weights(compteurBoucle) = 2000000;

            compteurBoucle = compteurBoucle + 1;
            source_nodes_idx(compteurBoucle) = boucle+1+largeur;
            destination_nodes_idx(compteurBoucle) = boucle;
            weights(compteurBoucle) = 2000000;

            compteurBoucle = compteurBoucle + 1;
            source_nodes_idx(compteurBoucle) = boucle+1;
            destination_nodes_idx(compteurBoucle) = boucle+largeur;
            weights(compteurBoucle) = 2000000;
        end
    end
```

Figure 4 : Code permettant de lier la grande majorité des pixels de l'image entre eux

Comme vous pouvez le remarquer, nos deux boucles ont comme indice de fin hauteur-1 et largeur-1. Nous avons choisi hauteur-1 car les pixels de la dernière ligne ne peuvent pas être reliés à des pixels qui ne sont pas présents dans l'image (hors limite). Nous avons choisi largeur-1 car les pixels tout à droite ne peuvent également pas être reliés à des pixels qui ne sont pas présents dans l'image (hors limite).

Pour expliquer, cette partie de code implémente une boucle double « **for** » qui parcourt l'image pour construire un graphe.

Les variables « source_nodes_idx », « destination_nodes_idx » et « weights » sont utilisées pour construire le graphe. La variable « compteurBoucle » est utilisée pour compter le nombre d'arêtes dans le graphe.

Pour chaque pixel de l'image (représenté par « **boucle** »), 4 arêtes sont ajoutées au graphe : une arête vers le pixel à droite, une arête vers le pixel à gauche (l'inverse de la

précédnte), une arête vers le pixel en dessous à droite et une arête vers le pixel en dessous à gauche. Les poids de ces arêtes sont déterminés par la variable « **weights** » et sont égaux à la valeur de « **E1** » pour la première arête et à 2000000 pour les autres arêtes.

Comme dit précédemment, il nous reste la dernière ligne tout en bas que n'avons pas fait car nous ne voulions pas liés des pixels au vide. Pour cela, nous avons dû faire un cas spécial :

```
for ligne = 1:largeur-1

    boucle = boucle + 1;

    compteurBoucle = compteurBoucle + 1;
    source_nodes_idx(compteurBoucle) = boucle;
    destination_nodes_idx(compteurBoucle) = boucle+1;
    weights(compteurBoucle) = E1(hauteur,ligne);


    compteurBoucle = compteurBoucle + 1;
    source_nodes_idx(compteurBoucle) = boucle+1;
    destination_nodes_idx(compteurBoucle) = boucle;
    weights(compteurBoucle) = 2000000;
end
```

Figure 5 : Code permettant de lier les pixels de la dernière ligne entre eux

Cette partie de code implémente une boucle « **for** » qui parcourt chaque ligne de la dernière colonne de l'image.

Pour chaque pixel de la dernière colonne de l'image (représenté par « **boucle** »), 2 arêtes sont ajoutées au graphe : une arête vers le pixel à droite et une arête vers le pixel de gauche. Les poids de ces arêtes sont déterminés par la variable « **weights** » et sont égaux à la valeur de « **E1** » pour la première arête et à 2000000 pour la seconde arête. Cela représente la dernière ligne de la figure de droite vu précédemment.

Finalement, nous pouvons voir sur la figure à gauche que tous les pixels du bord droit sont reliés au puit T Le poids allant des pixels du bord droit au puit T vaut l'infini c'est-à-dire une très grande valeur :

```
for colonne = 1:hauteur
    compteurBoucle = compteurBoucle + 1;
    source_nodes_idx(compteurBoucle) = (1+largeur)+((colonne-1)*largeur);
    destination_nodes_idx(compteurBoucle) = boucle;

    source_nodes_idx(compteurBoucle);
    weights(compteurBoucle) = 2000000;
end
```

Figure 6 : Code permettant de lier tous les pixels du bord droit au puit T

Puis nous pouvons facilement construire le graphe orienté G après avoir réalisé toutes ces arêtes via le code suivant :

```
    G = digraph(source_nodes_idx, destination_nodes_idx, weights); % Construire un
graphe à partir de l'im
    [mf,~,cs,ct] = maxflow(G, 1, boucle, 'searchtrees');
```

Figure 7 : code de construction du graphe G

Le code définit un graphe orienté en utilisant les fonctions « **digraph** » et « **maxflow** » :
- « **G = digraph(source_nodes_idx, destination_nodes_idx, weights**) » crée un graphe orienté à partir des nœuds source (**source_nodes_idx**), des nœuds de destination (**destination_nodes_idx**), et des poids (**weights**) des arcs entre les nœuds.
- « **maxflow (G, 1, boucle, 'searchtrees')** » calcule le flot maximum dans ce graphe orienté « **G** » en utilisant l'algorithme de « **searchtrees** ». Le premier argument « **1** » définit le nœud source, et le second argument « **boucle** » définit le nœud puit. La fonction « **maxflow** » retourne le flot maximum (**mf**), ainsi que les indices de coupe minimale pour les sommets source (**cs**) et les sommets de destination (**ct**).

1.3 Construire l'image binaire qui est une partition de G

Puis nous devons réaliser une partition binaire afin de séparer tous les éléments dans « **cs** » et tous les éléments dans « **ct** »

```
% PARTITION BINAIRE
n = numnodes(G);
partition = zeros(n,1);
for i = 1:n
    if ismember(i,cs)
        partition(i) = 1;
    elseif ismember(i,ct)
        partition(i) = 0;
    end
end
```

Figure 8 : Code permettant la création d'une partition binaire

Ce code implémente l'étape de partition binaire. Il utilise la fonction « **numnodes** » pour obtenir le nombre total de nœuds (points) dans le graphe G. Ensuite, il crée un tableau partition avec une longueur égale au nombre de nœuds et tous ses éléments sont initialisés à zéro.

Pour chaque nœud dans le graphe, le code vérifie s'il fait partie des nœuds source « **cs »** ou des nœuds de destination « **ct** » en utilisant la fonction « **ismember »**. Si le nœud est un nœud source, sa valeur dans le tableau partition sera définie sur 1. S'il est un nœud de destination, sa valeur sera définie sur 0. Ce processus permet de déterminer à quelle partition appartient chaque nœud du graphe.

1.4 Chercher la frontière entre S et T, la supprimer et recaller les deux parties

Maintenant notre objectif est d'identifier les indices de la frontière dans le partitionnement binaire obtenu à partir du graphe.

```
% LES INDICES DE LA FRONTIERE
indices = [];
for i = 1:n-1
    if partition(i) ~= partition(i+1)
         indices = [indices, i];
    end
end

% LES INDICES FILTRES DE LA FRONTIERE
longueurPartition = length(indices);
for i = 1:longueurPartition
    for j = 1:hauteur
        if(i < length(indices))
            if (indices(i) == (1+largeur*j))
                indices(i) = [];
            end
        end
    end
end

%LES INDICES FILTRES DE LA FRONTIERE PAR LIGNE
for i = 1:length(indices)
    if(i > 0)
       indices(i) = indices(i)-largeur*(i-1);
    end
end
```

Figure 9 : Code d'identification des indices de la frontière entre S et T

Ce code a pour objectif d'identifier les indices de la frontière dans le partitionnement binaire obtenu à partir du graphe.

Tout d'abord, le code parcourt tous les nœuds (sauf le dernier) du graphe et vérifie si la partition du nœud actuel est différente de celle du nœud suivant. Si c'est le cas, l'index du nœud actuel est ajouté à la liste « indices ».

Le code filtre ensuite les indices qui sont qui sont les premiers d'une ligne via la ligne « **if (indices(i) == (1+largeur*j))** »

Finalement, le code ajuste les indices pour refléter leur position pour chacune des lignes. C'est-à-dire, pour chaque index i, l'index est mis à jour en soustrayant « **largeur*(i-1)** ».

Avec les indices de la frontière entre S et T, nous pouvons recréer une nouvelle image en retirant tous ces indices et ainsi recaller les deux parties :

```
newImage = uint8(zeros(hauteur,largeur-1,3));

for i = 1:hauteur
compteurSuppressionElement = 0;
compteurBloquant = 0;
    for j = 1:largeur
        compteurSuppressionElement = compteurSuppressionElement + 1;

        if(compteurSuppressionElement ~= indices(i) && compteurBloquant == 0)
           newImage(i,j,:) = I(i,j,:);
        end

        if(compteurSuppressionElement == indices(i))
            compteurBloquant = 1;
        end

        if(compteurSuppressionElement ~= indices(i) && compteurBloquant == 1)
           newImage(i,j-1,:) = I(i,j,:);
        end
    end
end
```

Figure 10 : Code pour supprimer la frontière et recaller les deux parties

Ce code traite une image en couleurs (**I**) et génère une nouvelle image (**newImage**) en utilisant le tableau d'indices (**indices**). La boucle « **for** » externe parcourt les lignes de l'image et la boucle « **for** » interne parcourt les colonnes de l'image. Le **compteurSuppressionElement** est utilisé pour suivre la progression par rapport à l'indice de chaque ligne, tandis que le **compteurBloquant** est utilisé pour indiquer si l'indice correspondant a déjà été atteint.

Nous ajoutons chaque élément de l'image (**I**) dans le nouvelle image (**newImage**) sauf bien évidemment les éléments de la frontière.

Finalement, nous pouvons afficher notre nouvelle image (**newImage**) et la sauvegarder avec le même nom que l'image d'entrée :

```
imshow(newImage);

imwrite(newImage,'banc.jpg');
```

Figure 11 : Code pour afficher et sauvegarder la nouvelle image

1.5 Recommencer à l'étape 3 (« Construire le graphe G à l'aide de l'image E_1 ») jusqu'à atteindre la longueur d'image voulue

Pour recommencer, cela est très simple. Il suffit d'enregistrer notre image modifiée et de réaliser un boucle **while** qui a comme conditions de sortie le nombre d'itérations que nous souhaitons faire (donc le nombre de pixels que nous souhaitons réduire notre image) :

```
nbIterations = 100;

compteurIteration = 0;

while (compteurIteration < nbIterations)


%LA TOTALITE DU CODE VU PRECEDEMMENT


compteurIteration = compteurIteration + 1

end
```

Figure 12 : Code permettant de boucler notre programme afin de réduire l'image jusqu'à la longueur voulue

1.6 Analyse de la complexité et du temps d'exécution de l'algorithme

Ce code en Matlab utilise une boucle **while** pour exécuter un certain nombre d'itérations (ici 100) défini par la variable « **nbIterations** ». À chaque itération, l'image « **banc.png** » est lue (la qualité de l'image est mieux sauvegardée au format .png qu'au format .jpg qui compresse l'image), convertie en niveaux de gris s'il s'agit d'une image en couleur, puis le gradient de l'image en niveaux de gris est calculé.

Ensuite, des variables « **source_nodes_idx** », « **destination_nodes_idx** » et « **weights** » sont initialisées. Deux boucles for sont ensuite utilisées pour remplir ces variables en fonction de la taille de l'image (hauteur et largeur). Les valeurs des variables dépendent du gradient de l'image et sont utilisées pour créer un graphe (digraph) appelé « **G** ».

La complexité de ce code dépend de la taille de l'image. La lecture et la conversion de l'image en niveaux de gris sont relativement rapides et n'ont pas une grande influence sur la complexité totale. Le calcul de la norme L_1 du gradient peut prendre un peu plus de temps, mais reste gérable pour des images de taille raisonnable.

Les boucles for utilisées pour remplir les variables « **source_nodes_idx** », « **destination_nodes_idx** » et « **weights** » ont une complexité en $O(n^2)$, où n est la taille de l'image (hauteur x largeur). En conséquence, plus l'image est grande, plus ce processus sera long. La création du graphe avec la fonction digraph a également une complexité en $O(n^2)$, de sorte que la complexité totale de ce code est en $O(n^2)$. En résumé, la complexité de ce code dépend de la taille de l'image et est de l'ordre de n^2.

De plus, nous avons remarqué que la partition binaire prend du temps à s'exécuter (la majorité du temps d'exécution). En effet, dans chaque itération de la boucle for, la fonction **ismember** est appelée pour vérifier si l'élément i est présent dans l'un des deux ensembles **cs** ou **ct**. Si les tailles de ces ensembles sont très grandes, il peut prendre beaucoup de temps pour parcourir l'ensemble entier et trouver la correspondance. La complexité de la fonction **ismember** est en général en O(n) où n est la taille de l'ensemble, donc la complexité totale pour cette partie du code peut être en $O(n^2)$.

1.7 Analyse des résultats obtenus

1.7.1 Le banc

Initialement, nous avons cette image RGB de ce banc qui mesure 604 pixels en largeur et 290 pixels en hauteur :



Figure 13 : Image initial du banc à traiter

Nous avons réalisé une modification dans notre programme afin d'afficher par des lignes noires les frontières qui normalement devraient être supprimées. Voici donc une image du banc en niveau de gris après avoir appliquée 10 fois notre algorithme :

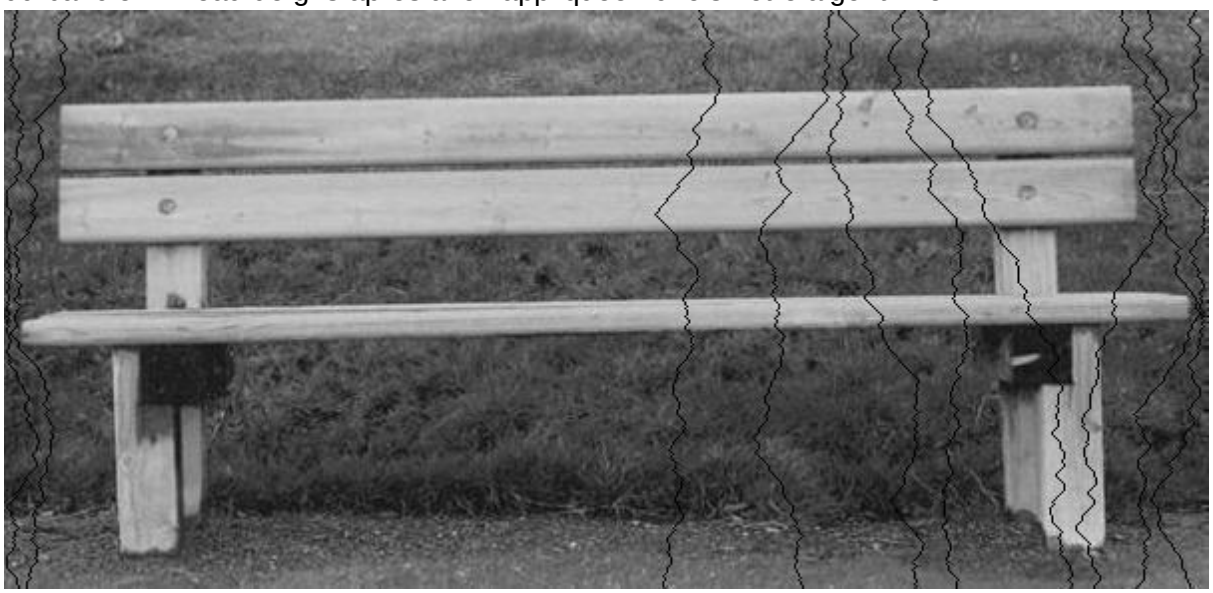


Figure 14 : Image du banc en niveau de gris après 10 itérations

Comme nous pouvons le voir, une grande partie des lignes se situent sur les bords ou au niveau du milieu du banc car se sont les éléments les moins importants présents sur l'image.

Voici maintenant l'image du banc après 70 itérations soit 70 fois où nous avons appliqué notre algorithme :

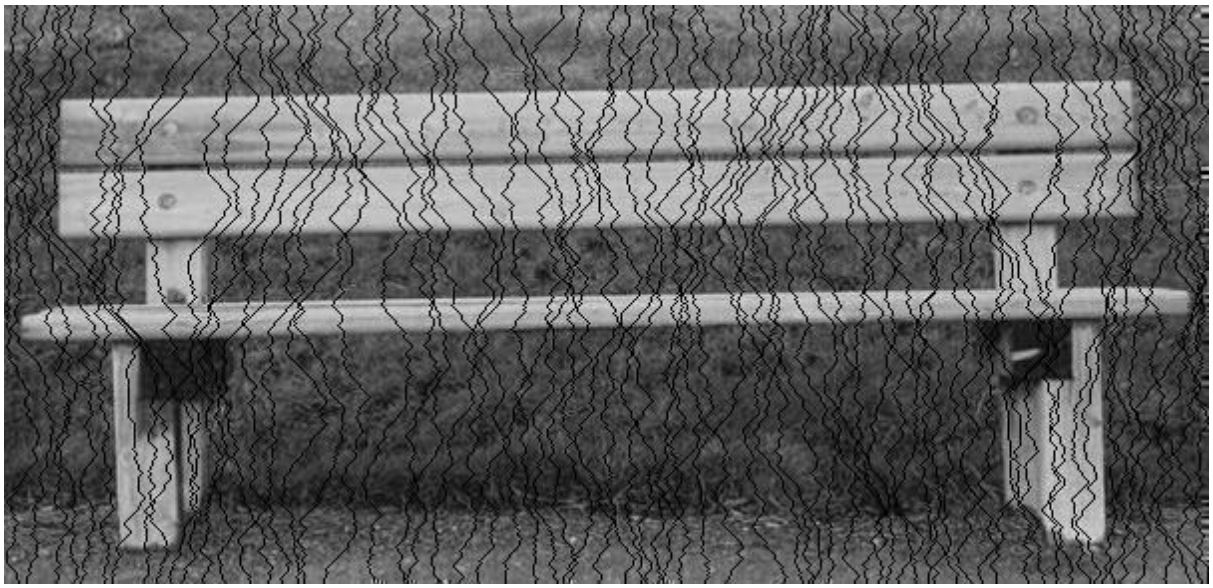


Figure 15 : Image du banc en niveau de gris après 70 itérations

Nous voyons maintenant que les lignes noirs qui correspond à la frontière entre S et T pour une itération semblent être sur la totalité de l'image

Voici donc le banc après 10 itérations de notre algorithme. Cette fois-ci nous supprimons réellement les pixels. Comme nous pouvons le voir, cela ne semble pas être visible à l'œil nu, trop peu de pixels ont été supprimé :



Figure 16 : Image du banc en niveau de gris après 10 itérations



Figure 17 : Image du banc en niveau de gris après 70 itérations



Figure 18 : Image du banc en niveau de gris après 100 itérations

Après 100 itérations, la taille de l'image n'est plus que de 504 par 290. Nous remarquons que l'espace à gauche et à droite du banc a été réduit et que les pieds du banc restent inchangés ou presque. La prochaine cible de l'algorithme sera principalement le milieu du banc.



Figure 19 : Image du banc en niveau de gris après 200 itérations



Figure 20 : Image du banc en niveau de gris après 300 itérations

Après 300 itérations, la taille de l'image n'est plus que de 304 par 290. Nous remarquons les pieds du banc se sont globalement rétrécis mais reste de taille conséquente. Globalement, l'image reste de bonne qualité sans trop d'aberrations visibles.



Figure 21 : Image du banc en niveau de gris après 400 itérations



Figure 22 : Image du banc en niveau de gris après 500 itérations

Après 500 itérations, la taille de l'image n'est plus que de 104 par 290. Nous remarquons des aberrations fortement visibles. Le banc a plutôt l'allure d'une chaise.

1.7.2 La plage

Initialement, nous avons cette image RGB de ce banc qui mesure 307 pixels en largeur et 230 pixels en hauteur :



Figure 23 : Image initial de la plage à traiter

Nous avons réalisé une modification dans notre programme afin d'afficher par des lignes noires les frontières qui normalement devraient être supprimées. Voici donc une image du banc en niveau de gris après avoir appliquée 50 fois notre algorithme :

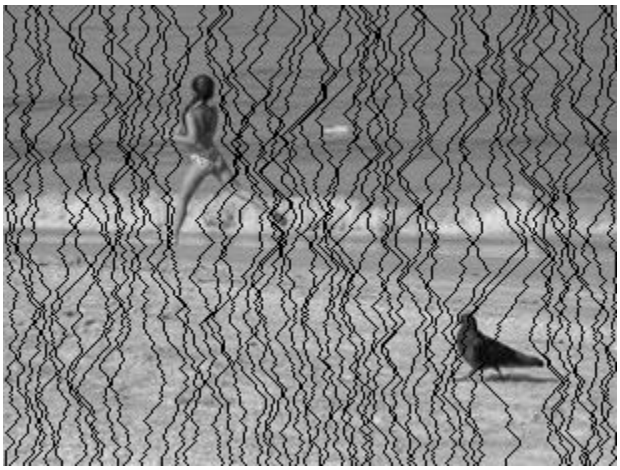


Figure 24 : Image de la plage en niveau de gris après 50 itérations

Voici maintenant l'image du banc après 50 itérations soit 50 fois où nous avons appliqué notre algorithme :



Figure 25 : Image de la plage en niveau de gris après 50 itérations



Figure 26 : Image de la plage en niveau de gris après 100 itérations



Figure 27 : Image de la plage en niveau de gris après 200 itérations



Figure 28 : Image de la plage en niveau de gris après 250 itérations

Nous remarquons qu'après chacune des différentes itérations, nous retrouvons toujours l'oiseau ainsi que la jeune fille. Cependant, le pied droit de la jeune fille a été supprimé (comme nous avons vu en cours). Cela est particulièrement flagrant après 200 itérations.

1.7.3 L'oiseau

L'oiseau étant intraitable dans les dimensions actuelles (1920 par 1080), nous avons décidé de le réduire au format 480 (1920/4) par 270 (1080/4). Voici l'image initial que nous avons traité :



Figure 29 : Image initial de l'oiseau à traiter

Nous avons réalisé une modification dans notre programme afin d'afficher par des lignes noires les frontières qui normalement devraient être supprimées. Voici donc une image du banc en niveau de gris après avoir appliquée 50 fois notre algorithme :



Figure 30 : Image de l'oiseau en niveau de gris après 50 itérations

Voici maintenant l'image du banc après 50 itérations soit 50 fois où nous avons appliqué notre algorithme :



Figure 31 : Image de l'oiseau en niveau de gris après 50 itérations



Figure 32 : Image de l'oiseau en niveau de gris après 100 itérations



Figure 33 : Image de l'oiseau en niveau de gris après 200 itérations



Figure 34 : Image de l'oiseau en niveau de gris après 300 itérations



Figure 35 : Image de l'oiseau en niveau de gris après 400 itérations

2. Seam carving amélioré

Introduction

L'algorithme de Chan-Vese est un modèle de segmentation d'images qui se concentre sur la séparation des différents objets dans une image en identifiant les différences de contraste entre les régions. Il utilise une méthode de minimisation d'énergie pour déterminer les bords des objets. Cette technique peut être utile pour éviter le traversal des objets d'intérêt dans l'image lors de la technique de seam carving.

Le choix de Chan-Vese pour ce problème peut être justifié en raison de sa capacité à identifier les différences de contraste entre les régions dans une image, ce qui peut aider à séparer les objets d'intérêt des autres régions de l'image. Les paramètres peuvent être ajustés pour améliorer la performance du modèle. Les résultats peuvent être montrés sur un échantillon d'images choisi pour illustrer les avantages de cette technique de segmentation.

2.1 Choix des paramètres λ_1 et λ_2

Le modèle de Chan-Vese utilise deux paramètres principaux à savoir :
– λ_1 qui représente le poids des pixels à l'intérieur de la région
– λ_2 qui représente le poids des pixels à l'extérieur de la région.

Les valeurs de ces paramètres déterminent comment l'algorithme pondère les pixels à l'intérieur et à l'extérieur de la région dans le calcul de l'énergie.

Comme notre objectif est de maximiser la distinction entre l'objet d'intérêt et le fond de l'image, une valeur élevée pour λ_1 et une valeur faible pour λ_2 doivent être choisies. En effet, λ_1 représente la force de la contrainte pour préserver l'objet d'intérêt et λ_2 représente la force de la contrainte pour éliminer les fonds ou les parties inutiles de l'image. Cependant, le choix exact des valeurs dépendra de chaque image et de son objet d'intérêt spécifique.

3 Annexe (code Matlab)

```
nbIterations = 100;

compteurIteration = 0;



while (compteurIteration < nbIterations)

    I = imread('banc.png'); % Charger l'image
    if ndims(I) == 3
        I = rgb2gray(I); % Convertir en niveaux de gris
    end

    [hauteur, largeur] = size(I);
    % Calcul de la norme L1 du gradient
    [gradientX,gradientY] = imgradientxy(I);
    E1 = abs(gradientX) + abs(gradientY);



    source_nodes_idx = [hauteur+((hauteur-1)*(largeur-1))*4+(2*(largeur-
1))+hauteur];
    destination_nodes_idx = [hauteur+((hauteur-1)*(largeur-1))*4+(2*(largeur-
1))+hauteur];
    weights = [hauteur+((hauteur-1)*(largeur-1))*4+(2*(largeur-1))+hauteur];

    boucle = 1;
    compteurBoucle = 0;

    for colonne = 1:hauteur
        compteurBoucle = compteurBoucle + 1;
        source_nodes_idx(compteurBoucle) = boucle;
        destination_nodes_idx(compteurBoucle) = (boucle+1)+((colonne-1)*largeur);
        weights(compteurBoucle) = 2000000;
    end


    for colonne = 1:hauteur-1

        if colonne > 1
           boucle = boucle + 1;
        end

        for ligne = 1:largeur-1

            boucle = boucle + 1;

            compteurBoucle = compteurBoucle + 1;
            source_nodes_idx(compteurBoucle) = boucle;
            destination_nodes_idx(compteurBoucle) = boucle+1;
            weights(compteurBoucle) = E1(colonne,ligne);

            compteurBoucle = compteurBoucle + 1;
            source_nodes_idx(compteurBoucle) = boucle+1;
            destination_nodes_idx(compteurBoucle) = boucle;
            weights(compteurBoucle) = 2000000;

            compteurBoucle = compteurBoucle + 1;
            source_nodes_idx(compteurBoucle) = boucle+1+largeur;
            destination_nodes_idx(compteurBoucle) = boucle;
            weights(compteurBoucle) = 2000000;
```

```
            compteurBoucle = compteurBoucle + 1;
            source_nodes_idx(compteurBoucle) = boucle+1;
            destination_nodes_idx(compteurBoucle) = boucle+largeur;
            weights(compteurBoucle) = 2000000;
        end
    end

    boucle = boucle + 1;

    for ligne = 1:largeur-1

        boucle = boucle + 1;

        compteurBoucle = compteurBoucle + 1;
        source_nodes_idx(compteurBoucle) = boucle;
        destination_nodes_idx(compteurBoucle) = boucle+1; % Peux etre (boucle-
1+largeur)
        weights(compteurBoucle) = E1(hauteur,ligne);

        compteurBoucle = compteurBoucle + 1;
        source_nodes_idx(compteurBoucle) = boucle+1;
        destination_nodes_idx(compteurBoucle) = boucle; % Peux etre (boucle-
1+largeur)
        weights(compteurBoucle) = 2000000;
    end

    boucle = largeur * hauteur + 2;

    for colonne = 1:hauteur
        compteurBoucle = compteurBoucle + 1;
        source_nodes_idx(compteurBoucle) = (1+largeur)+((colonne-1)*largeur);
        destination_nodes_idx(compteurBoucle) = boucle;

        source_nodes_idx(compteurBoucle);
        weights(compteurBoucle) = 2000000;
    end

    G = digraph(source_nodes_idx, destination_nodes_idx, weights); % Construire un
graphe à partir de l'im
    [mf,~,cs,ct] = maxflow(G, 1, boucle, 'searchtrees');
    %mf
    %cs
    %ct

    % PARTITION BINAIRE
    n = numnodes(G);
    partition = zeros(n,1);
    for i = 1:n
        if ismember(i,cs)
            partition(i) = 1;
        elseif ismember(i,ct)
            partition(i) = 0;
        end
    end


    % LES INDICES DE LA FRONTIERE
    indices = [];
    for i = 1:n-1
        if partition(i) ~= partition(i+1)
             indices = [indices, i];
        end
    end

    % LES INDICES FILTRES DE LA FRONTIERE
    longueurPartition = length(indices);
    for i = 1:longueurPartition
```

```
        for j = 1:hauteur
            if(i < length(indices))
                if (indices(i) == (1+largeur*j))
                    indices(i) = [];
                end
            end
        end
    end

    %LES INDICES FILTRES DE LA FRONTIERE PAR LIGNE
    for i = 1:length(indices)
        if(i > 0)
           indices(i) = indices(i)-largeur*(i-1);
        end
    end

    newImage = uint8(zeros(hauteur,largeur-1,3));

    for i = 1:hauteur
    compteurSuppressionElement = 0;
    compteurBloquant = 0;
        for j = 1:largeur
            compteurSuppressionElement = compteurSuppressionElement + 1;

            if(compteurSuppressionElement ~= indices(i) && compteurBloquant == 0)
               newImage(i,j,:) = I(i,j,:);
            end

            if(compteurSuppressionElement == indices(i))
                compteurBloquant = 1;
            end

            if(compteurSuppressionElement ~= indices(i) && compteurBloquant == 1)
               newImage(i,j-1,:) = I(i,j,:);
            end
        end
    end


    %AFFICHAGE DES LIGNES NOIRS
%       for i = 1:length(indices)
%           x = i;
%           y = indices(i)-largeur*(i-1);
%           I(x,y) = 0;
%       end

    compteurIteration = compteurIteration + 1
    imshow(newImage);
    imwrite(newImage,'banc.png');

end
```